

RL Nominal Controller — Refresher Notes

How the RL part works, and how the code is organized

1. How the RL Training Works (SAC-Lagrangian)

- Three roles, five networks: conceptually there are three functional pieces — the **policy (actor)**, the **reward critic**, and the **cost critic**. In the actual implementation each critic (reward and cost) is a small ensemble of 2 networks for training stability, so it's 5 networks total, but 3 roles is the right mental model.
- **Policy (actor)**: input is the state (position + heading of the car), output is the action (steering input). This is the network that becomes the 'nominal controller' once trained.
- **Reward critic**: estimates the expected future reward for a given state-action pair. The reward itself is simply *minus the distance to the goal* — continuous, and always pulling the policy toward the goal.
- **Cost critic**: estimates the expected future safety cost for a given state-action pair. The cost signal is **hybrid**: it is exactly 0 when the car is farther than a certain margin from the obstacle, and once inside that margin it becomes continuous, growing as *minus the distance to the obstacle* (the closer you get, the higher the cost).
- **Lagrange multiplier (λ)**: a single learned scalar (not a neural network) that decides how much the cost critic's signal should push back on the policy. The policy's loss is: maximize reward, minus λ times cost. So λ is the 'safety sensitivity dial.'
- **λ is computed by solving an optimization problem** — a primal-dual pair: the actor solves the primal problem (maximize reward subject to cost $\leq$ budget), and λ is updated by gradient ascent on the dual problem. If the measured cost is above the allowed budget, λ increases (more safety pressure); if the cost is below budget, λ decreases (less safety pressure, more freedom to chase reward). λ is clipped to stay non-negative.
- **Why we set the allowed cost budget high**: our nominal controller is meant to be purely task-focused, with no built-in safety awareness, so that our CBVF/CP safety filter has real unsafe behavior to correct. So we set the cost budget very high (effectively disabling the safety constraint). This means the measured cost never exceeds the budget, so λ has no reason to grow and settles near zero on its own — the safety term drops out of the policy's loss, and training becomes equivalent to plain reward-only SAC.

2. How the Code Is Organized (by folder)

redexp/envs/

- Defines the environment: the Dubins car dynamics, the reward function, the hybrid cost function, and episode termination conditions.

- RL itself is model-free (the policy never sees the dynamics equations directly — it only learns from sampled transitions), but we still need the explicit dynamics model here to run the simulation, and later for the CBVF computation, which is model-based and needs the dynamics in closed form.

jaxrl5/

- Contains the generic RL algorithm machinery: the neural network building blocks (actor, critics), the action distribution (squashed to valid steering range), the replay buffer, and the two learning algorithm variants:
 - — `sac/`: plain SAC, pure reward-maximizing, no safety cost at all.
 - — `sac_lag/`: SAC-Lagrangian, same core algorithm plus the cost critic and λ described above.

train/

- Runs the actual training loop: takes environment steps, updates all the networks, periodically evaluates the policy, and periodically saves checkpoints.
- The saved checkpoints (the actor network's trained weights) ARE the nominal controller.
- This folder also contains the loader/wrapper that turns a saved checkpoint into a callable controller object — this is the object that gets handed off to the CBVF/CP safety filter as 'the nominal controller to watch and correct.'

conformal_shield.py (top level)

- Not part of RL itself — this is where the trained RL nominal policy gets consumed: it loads the checkpoint, runs it inside the environment, and wraps it with the CBVF/CP safety shield for comparison.

3. The Flow, in One Line

Dynamics defined in **redexp** → learned via an algorithm in **jaxrl5** → trained and checkpointed in **train/** → the resulting policy object (also packaged in **train/**) is handed to **conformal_shield.py** as the nominal controller the safety filter watches and, when needed, overrides.